

OOP Assignment Report

Household Energy Consumption Tracker

WEN SIHAO (0379856)

ABDULLAH AHONAF FAHOMID ZIM (0390367)

Su Tingwei (0384175) To Tsun Kai (0373689)

MOHAMMAD SHAHNAWAZ (0389395)

July 6, 2026

Table of contents

1	Introduction	2
1.1	Background and Motivation	2
1.2	Connection to SDG 7	2
1.3	Main Objectives	3
2	Problem Description	3
2.1	Problem Statement	3
2.2	Why This Matters and Our Computational Solution	3
2.3	Target Audience	4
3	Application Design	4
3.1	Object-Oriented Design Principles	4
3.1.1	Encapsulation (Data Protection)	4
3.1.2	Inheritance (Code Reuse and Hierarchy)	5
3.1.3	Polymorphism (Dynamic Behaviour) – Key Feature	6
3.1.4	Class Interaction (Layered Architecture, Composition and Separation of Concerns)	7
3.1.5	Class Diagram	9
4	Implementation	12
4.1	Programming Language and Environment	12
4.2	Key Code Snippets Showing OOP Features	12
4.2.1	Encapsulation	13
4.2.2	Inheritance	13
4.2.3	Polymorphism (Dynamic Behaviour)	14
4.3	Execution and User Input/Output Handling	15
5	Output and Testing	17
5.1	Testing Strategy and Sample Inputs	17

5.2	Console Results (Sample Output)	20
5.3	Issues, Limitations, and Debugging	23
5.3.1	Bug 1: The Scanner <code>.nextInt()</code> Skipping Problem	23
5.3.2	Bug 2: <code>toString()</code> Showing Wrong Energy for Subclasses	24
5.3.3	Limitation: No Data Persistence	24
6	Discussion and Reflection	24
6.1	Reflection on OOP	24
6.2	What We Learned	25
6.3	Future Improvements	25
7	Conclusion	26
8	References	26

Course Name: ITS63304 Object-Oriented Programming

Group Name: Group 3

Program: Bachelor of Software Engineering/Bachelor of Computer Science

Semester: Semester-1 April 2026

Instructor: SHAKEEL AHMED

1 Introduction

1.1 Background and Motivation

In Malaysia, the weather is hot and humid for most of the year. Because of this, almost every household relies heavily on air conditioning and fans to stay comfortable. The problem is that running these appliances for long hours every day leads to very high electricity bills from TNB. Most people only realise how much energy they are using when the bill arrives at the end of the month – and by then, it is too late to do anything about it.

This is what motivated our team to build the **Household Energy Consumption Tracker**. We wanted to create a simple Java program that helps users understand how much energy each of their appliances is using, so they can make better decisions about their energy habits before the bill comes.

1.2 Connection to SDG 7

Our project is linked to **United Nations Sustainable Development Goal 7: Affordable and Clean Energy**. SDG 7 aims to make sure everyone has access to affordable, reliable, and modern energy. One way to support this goal is by reducing unnecessary energy waste at the household level. Our program helps with this by tracking how much energy each appliance uses and showing users which appliances are consuming the most. It also encourages the use of energy-efficient options like LED lights, which our program simulates with a 20% energy discount.

1.3 Main Objectives

Our program has four main objectives:

1. **Record and calculate household energy consumption** – Allow users to add different types of appliances (general, lighting, cooling) and automatically calculate how much energy each one uses in kilowatt-hours (kWh).
 2. **Identify high-energy-consuming appliances** – Let users set an energy threshold and filter out the appliances that use more than that amount, so they know which ones to target for energy saving.
 3. **Raise energy-saving awareness** – Show users how switching to LED lights and setting air conditioners to 24 degrees or above can reduce their energy usage, supporting the goals of SDG 7.
 4. **Estimate cost and average usage** – Calculate the total energy, the average usage per appliance, and the estimated electricity cost in Malaysian Ringgit (RM) using the TNB domestic tariff of RM 0.218 per kWh for the first 200 kWh block (Tenaga Nasional Berhad [TNB], n.d.), so users can see the real financial impact of their consumption.
-
-

2 Problem Description

2.1 Problem Statement

Every household in Malaysia receives a monthly electricity bill from TNB. But the bill only shows one total number – it does not break down how much each appliance used. So most people have no idea whether it is their air conditioner, their old refrigerator, or their water heater that is eating up most of the electricity. Without this information, they cannot make smart decisions about where to cut back.

2.2 Why This Matters and Our Computational Solution

High energy consumption is not just a money problem – it is also an environmental one. The more electricity we waste, the more fossil fuels get burned to generate it. To solve this, our team built a Java program that takes the power rating (in watts) and usage duration (in hours) of each appliance and calculates its energy consumption using the formula: **Energy (kWh) = Power (W) x Duration (h) / 1000**. This turns something abstract (“I think my AC uses a lot of power”) into a concrete number that users can compare and act on.

2.3 Target Audience

Our program is designed for three types of users:

- **Regular households** – families who want to understand and reduce their monthly electricity bills.
 - **University students living off-campus** – students who rent rooms or apartments and share utility bills, so knowing which appliance uses the most helps split costs fairly.
 - **Environmentally conscious individuals** – people who care about sustainability and want to track their carbon footprint by monitoring energy usage.
-
-

3 Application Design

3.1 Object-Oriented Design Principles

In this project, our team applied three main OOP principles: **Encapsulation**, **Inheritance**, and **Polymorphism**. We also used the **Composition** pattern to keep our code organized. Below we explain how we used each one, with references to our actual source code.

3.1.1 Encapsulation (Data Protection)

We used Encapsulation to make sure that the data inside our objects is safe from being changed incorrectly. In every class we wrote, all instance variables are set to **private**. This means no other class can directly access or change these values. If they want to read or update a value, they have to go through our public **getter** and **setter** methods.

What makes our setter methods different from basic ones is that we added **data validation** inside them. We did not just assign values – we checked them first. Here is a summary of what each setter checks:

Table 1: Setter Methods

Class	Setter Method	What it checks
Appliance	<code>setApplianceName(String)</code>	Makes sure the name is not null or empty. Throws <code>IllegalArgumentException</code> if it is.
Appliance	<code>setPowerRatingInWatts(double)</code>	Makes sure the power is not negative (must be ≥ 0).
Appliance	<code>setUsageDurationInHours(double)</code>	Makes sure the duration is not negative (must be ≥ 0).

Class	Setter Method	What it checks
CoolingAppliance	setTemperatureSetting(int)	Only allows temperatures between 16 and 30 degrees Celsius.
Household	setHouseholdName(String)	Makes sure the household name is not null or empty.
EnergyUsageRecord	setEnergyConsumedInKWh(double)	Makes sure energy values are not negative.

One thing our group decided early on is that the **constructors should call these setter methods** instead of assigning values directly. For example, in our `Appliance.java`:

```
public Appliance(String applianceName, double powerRatingInWatts, double
    ↪ usageDurationInHours) {
    setApplianceName(applianceName);           // Validation applied here
    setPowerRatingInWatts(powerRatingInWatts); // Validation applied here
    setUsageDurationInHours(usageDurationInHours); // Validation applied here
}
```

We did it this way so that the validation runs no matter how the object is created – whether it is through the constructor or later through a setter. This way, the object always protects its own data.

3.1.2 Inheritance (Code Reuse and Hierarchy)

Our team set up a parent-child relationship between the classes using the **extends** keyword. The structure looks like this:

```
Appliance          (Parent / Superclass)
|-- LightAppliance (Child / Subclass)
|-- CoolingAppliance (Child / Subclass)
```

What the child classes get from the parent:

The `Appliance` class has three shared attributes that both subclasses inherit automatically:

- `applianceName` (String) – the name of the appliance
- `powerRatingInWatts` (double) – how much power it uses in watts
- `usageDurationInHours` (double) – how many hours it is used

It also has a base method `calculateEnergyConsumption()`. This method uses the formula: Energy (kWh) = Power (W) x Duration (h) / 1000.

What the child classes add on their own:

Each child class has its own extra attribute:

- `LightAppliance` adds boolean `isLED` – this tells us if the light is an energy-saving LED or not.

- `CoolingAppliance` adds `int temperatureSetting` – this is the AC temperature in degrees Celsius.

How we used `super()` in the constructors:

In both child class constructors, we call `super()` to pass the shared attributes up to the parent. This way, the parent's setter validation still runs, and we don't copy-paste the same checks. Here is an example from `LightAppliance.java`:

```
// In LightAppliance.java
public LightAppliance(String applianceName, double powerRatingInWatts,
                      double usageDurationInHours, boolean isLED) {
    super(applianceName, powerRatingInWatts, usageDurationInHours); // Delegates
    ↪ to Appliance
    this.isLED = isLED;
}
```

Because of this IS-A relationship (a `LightAppliance` IS-A `Appliance`), we can store subclass objects in a parent-type variable or collection. This turns out to be very important for polymorphism, which we explain next.

3.1.3 Polymorphism (Dynamic Behaviour) – Key Feature

This is probably the most important OOP feature in our project. Both `LightAppliance` and `CoolingAppliance` **override** the `calculateEnergyConsumption()` method from the parent class, each with their own special logic:

- **LightAppliance:** If `isLED` is `true`, we multiply the base energy by **0.8**. This gives a 20% saving, which simulates how LED lights are more efficient in real life.
- **CoolingAppliance:** If `temperatureSetting` is below **24 degrees**, we multiply the base energy by **1.2**. This adds a 20% penalty because lower AC temperatures use more power.

Both of these overridden methods first call `super.calculateEnergyConsumption()` to get the base value, and then apply their own modifier on top.

How we used Dynamic Polymorphism in `EnergyCalculationService.calculateTotalEnergy()`:

In the `Household` entity class, we store all the appliances in an `ArrayList<Appliance>`. Notice that the list type is the **superclass** – `Appliance`, not `LightAppliance` or `CoolingAppliance`. So when we add a `LightAppliance` object into this list, it gets **upcast** to `Appliance` automatically.

The actual calculation lives in a separate class, `EnergyCalculationService` (in our `service` package). When we want to calculate the total energy for the whole household, the service just loops through the household's appliances with a basic `for` loop:

```
// In EnergyCalculationService.java - calculateTotalEnergy(Household household)
for (int i = 0; i < household.getApplianceCount(); i++) {

    // Step 1 - UPCASTING: access via superclass reference
    Appliance currentAppliance = household.getAppliance(i);

    // Step 2 - DYNAMIC DISPATCH: JVM calls the correct overridden method
    //          based on the ACTUAL type of the object at runtime
    double energy = currentAppliance.calculateEnergyConsumption();

    totalEnergy = totalEnergy + energy;
}
```

Here is what happens: even though `currentAppliance` is declared as type `Appliance`, the actual object behind it might be a `LightAppliance` or a `CoolingAppliance`. When we call `calculateEnergyConsumption()`, Java does not just run the parent version. Instead, the JVM looks at the **actual type** of the object at runtime and picks the right overridden version. This is called **dynamic method dispatch**. So:

- If the object is actually a `LightAppliance` with LED on, the LED-discounted version runs.
- If it is a `CoolingAppliance` set below 24 degrees, the penalty version runs.
- If it is just a plain `Appliance`, the base formula runs.

The nice thing about this is that we only need **one loop** to handle all different types. We do not need any `if-else` or `instanceof` to check what type each appliance is before calculating. The JVM handles it for us. This also means if we ever add a new subclass in the future (like a `HeatingAppliance`), we would not need to change this loop at all – it would just work.

3.1.4 Class Interaction (Layered Architecture, Composition and Separation of Concerns)

Our team organized the 11 classes into a **layered architecture** with 5 packages, using **Composition** (HAS-A relationships) instead of putting everything in one or two big classes. Each layer has exactly one kind of responsibility:

```
src/com/sdg/energytracker/
|-- main/      EnergyTrackerApp      - program entry point only
|-- controller/ EnergyTrackerController - menu loop, dispatches user choices
|-- entity/    Appliance, LightAppliance,
|              CoolingAppliance,
|              Household, EnergyUsageRecord - the data model
|-- service/   EnergyCalculationService,
|              ReportService         - the business logic
|-- utilities/ InputReader, ConsoleUI - reusable input/output helpers
```

The flow between the layers looks like this (similar to the MVC pattern):

```

EnergyTrackerApp (main)
    | creates
    v
EnergyTrackerController --uses--> InputReader / ConsoleUI (utilities)
    | delegates
    v
EnergyCalculationService / ReportService (service)
    | operate on
    v
Household --owns--> ArrayList<Appliance> (entity)

```

How the layers work together (one full round of “view energy report”):

1. When the program starts, `EnergyTrackerApp.main()` creates one object from each layer – the `Household` (entity), the `InputReader` (utility), and the two services – and supplies them to the `EnergyTrackerController` through its constructor (constructor injection). Then it calls the controller’s `run()` method and hands over control.
2. The controller HAS-A `Household`, an `InputReader`, an `EnergyCalculationService`, and a `ReportService` (Composition). It reads the user’s menu choice through `inputReader.readInt(...)`.
3. When the user wants to add an appliance, the controller collects the input via `InputReader`, creates the right subclass object, and calls `household.addAppliance(newAppliance)`. The `Household` entity handles the storage; it never does calculations.
4. When the user wants to see the energy report, the controller calls `calculationService.recordAllEnergyUsage()` and then prints `reportService.generateEnergyReport(household)`. The calculation and the text formatting are done by two different service classes.
5. Same for the high-energy filter – the controller calls `reportService.generateHighEnergyReport(household, threshold)`, and the `ReportService` internally delegates the filtering to the `EnergyCalculationService` (composition between two services).

We did it this way because we wanted each class to have only one job. This is the **Single Responsibility Principle (SRP)**:

Table 2: Layered Architecture

Layer / Class	What it does
main/ <code>EnergyTrackerApp</code>	Program entry point – creates the objects from the different layers, wires them into the controller, and starts it.
controller/ <code>EnergyTrackerController</code>	Runs the main loop and dispatches each user choice to the right collaborator.
entity/ <code>Appliance</code> and subclasses	Hold data and calculate individual energy consumption.
entity/ <code>Household</code>	Stores the household name, appliance list, and records (pure data management).
entity/ <code>EnergyUsageRecorder</code>	Logs energy data with a timestamp.
service/ <code>EnergyCalculationService</code>	Does the business logic: total, average, cost, high-energy filtering.
service/ <code>ReportService</code>	Formats the calculation results into human-readable text reports.
utilities/ <code>InputReader</code>	Robust, validated console input (never crashes on bad input).

Layer / Class	What it does
utilities/ConsoleUI	Banners, menu, separators – consistent console output.

So no class ever does two kinds of work: the entity classes never calculate totals, the services never read user input, and the controller never contains business formulas. Besides making the code cleaner, this also made teamwork much easier – each of the five group members owns a complete, clearly separated part of the system (see `TEAM_CONTRIBUTIONS.md`).

3.1.5 Class Diagram

```
classDiagram
class Appliance {
    -String applianceName
    -double powerRatingInWatts
    -double usageDurationInHours
    +Appliance()
    +Appliance(String, double, double)
    +getApplianceName() String
    +setApplianceName(String) void
    +getPowerRatingInWatts() double
    +setPowerRatingInWatts(double) void
    +getUsageDurationInHours() double
    +setUsageDurationInHours(double) void
    +calculateEnergyConsumption() double
    +toString() String
}

class LightAppliance {
    -boolean isLED
    +LightAppliance()
    +LightAppliance(String, double, double, boolean)
    +getIsLED() boolean
    +setIsLED(boolean) void
    +calculateEnergyConsumption() double
    +toString() String
}

class CoolingAppliance {
    -int temperatureSetting
    +CoolingAppliance()
    +CoolingAppliance(String, double, double, int)
    +getTemperatureSetting() int
    +setTemperatureSetting(int) void
    +calculateEnergyConsumption() double
}
```

```

        +toString() String
    }

class EnergyUsageRecord {
    -int recordId
    -Appliance appliance
    -double energyConsumedInKWh
    -LocalDateTime recordDateTime
    +EnergyUsageRecord()
    +EnergyUsageRecord(Appliance)
    +getRecordId() int
    +getAppliance() Appliance
    +setAppliance(Appliance) void
    +getEnergyConsumedInKWh() double
    +setEnergyConsumedInKWh(double) void
    +getRecordDateTime() LocalDateTime
    +setRecordDateTime(LocalDateTime) void
    +generateReport() String
    +toString() String
}

class Household {
    -String householdName
    -ArrayList~Appliance~ appliances
    -ArrayList~EnergyUsageRecord~ records
    +Household()
    +Household(String)
    +getHouseholdName() String
    +setHouseholdName(String) void
    +addAppliance(Appliance) void
    +getAppliance(int) Appliance
    +getApplianceCount() int
    +getRecords() ArrayList
    +toString() String
}

class EnergyTrackerApp {
    +main(String[]) void$
}

class EnergyTrackerController {
    -Household household
    -InputReader inputReader
    -EnergyCalculationService calculationService
    -ReportService reportService
    +EnergyTrackerController()
    +EnergyTrackerController(Household, InputReader, EnergyCalculationService,
↪ ReportService)
    +run() void

```

```

        -handleAddAppliance() void
        -handleEnergyReport() void
        -handleHighEnergyReport() void
    }

class EnergyCalculationService {
    +double TARIFF_RATE_RM_PER_KWH$
    +calculateTotalEnergy(Household) double
    +calculateAverageEnergy(Household) double
    +calculateEstimatedCost(Household) double
    +getHighEnergyAppliances(Household, double) ArrayList
    +recordAllEnergyUsage(Household) void
}

class ReportService {
    -EnergyCalculationService calculationService
    +ReportService()
    +ReportService(EnergyCalculationService)
    +generateEnergyReport(Household) String
    +generateHighEnergyReport(Household, double) String
}

class InputReader {
    -Scanner scanner
    +InputReader()
    +readInt(String, int, int) int
    +readDouble(String) double
    +readYesNo(String) boolean
    +readNonEmptyString(String, String) String
    +close() void
}

class ConsoleUI {
    +String SEPARATOR_THICK$
    +String SEPARATOR_MEDIUM$
    +String SEPARATOR_THIN$
    -ConsoleUI()
    +printWelcomeBanner() void$
    +printMenu() void$
    +printExitBanner() void$
    +printNoAppliancesInfo() void$
    +printUnexpectedError(String) void$
}

Appliance <|-- LightAppliance : Inheritance
Appliance <|-- CoolingAppliance : Inheritance
Household *-- Appliance : Composition
Household *-- EnergyUsageRecord : Composition
EnergyUsageRecord o-- Appliance : Association

```

```

EnergyTrackerApp ..> EnergyTrackerController : creates
EnergyTrackerController *-- Household : Composition
EnergyTrackerController *-- InputReader : Composition
EnergyTrackerController *-- EnergyCalculationService : Composition
EnergyTrackerController *-- ReportService : Composition
EnergyTrackerController ..> ConsoleUI : Uses
ReportService o-- EnergyCalculationService : Delegation
EnergyCalculationService ..> Household : Operates on
ReportService ..> Household : Operates on

```

4 Implementation

4.1 Programming Language and Environment

Our team chose **Java (JDK 8)** to build this project. We picked Java mainly because it has built-in support for OOP. Everything in Java is based on classes and objects, so it was a natural fit for an OOP assignment. Besides that, Java has strict type checking at compile time, which helped us catch bugs early. It also runs on any platform with a JVM, so any group member could compile and test the code on their own machine.

We used **Visual Studio Code** with the Java Extension Pack as our IDE. For version control, we used **Git** and hosted the repository on **GitHub** so that all five members could work together. We compiled the code using the `javac` command in the terminal.

Component	Detail
Programming Language	Java (JDK 8)
IDE	Visual Studio Code with the Java Extension Pack
Version Control	Git, hosted on GitHub
Build Method	Command-line compilation using <code>javac</code>
Operating System	Windows

4.2 Key Code Snippets Showing OOP Features

Below we show some key parts of our code that demonstrate the three OOP principles. We only included the relevant lines – we did not put full class code here.

4.2.1 Encapsulation

From `Appliance.java` – private attributes and setter validation:

```
// All attributes are declared 'private' -- other classes cannot touch them
↪ directly
private String applianceName;
private double powerRatingInWatts;
private double usageDurationInHours;

// Setter with data validation -- checks before assigning
public void setPowerRatingInWatts(double powerRatingInWatts) {
    if (powerRatingInWatts < 0) {
        throw new IllegalArgumentException(
            "Power rating cannot be negative. Received: " +
            ↪ powerRatingInWatts);
    }
    this.powerRatingInWatts = powerRatingInWatts;
}

public void setApplianceName(String applianceName) {
    if (applianceName == null || applianceName.trim().isEmpty()) {
        throw new IllegalArgumentException("Appliance name cannot be null or
            ↪ empty.");
    }
    this.applianceName = applianceName.trim();
}
```

What this shows: We declared all our variables as `private`, so nothing outside the class can directly read or change them. The only way to update them is through our setter methods. And our setters do not just blindly accept any value – they check the input first. For example, `setPowerRatingInWatts()` checks if the number is negative. If someone tries to pass in `-100`, it throws an `IllegalArgumentException` instead of accepting bad data. We also made our constructors call these setters (like `setApplianceName(applianceName)` instead of `this.applianceName = applianceName`), so the validation always runs, no matter how the object gets created.

4.2.2 Inheritance

From `LightAppliance.java` – using `extends` and `super()`:

```
public class LightAppliance extends Appliance {

    // Extra private attribute only for this subclass
    private boolean isLED;

    // Constructor -- passes shared attributes to parent using super()
```

```

    public LightAppliance(String applianceName, double powerRatingInWatts,
                           double usageDurationInHours, boolean isLED) {
        super(applianceName, powerRatingInWatts, usageDurationInHours);
        this.isLED = isLED;
    }

    // Override -- different calculation for light appliances
    @Override
    public double calculateEnergyConsumption() {
        double baseEnergy = super.calculateEnergyConsumption();
        if (isLED) {
            return baseEnergy * 0.8; // LED lights save 20% energy
        } else {
            return baseEnergy;
        }
    }
}

```

What this shows: We used the `extends` keyword to make `LightAppliance` a child of `Appliance`. This means `LightAppliance` automatically gets all the parent's attributes and methods – we did not need to rewrite them. In the constructor, we used `super()` to send the shared data up to the parent's constructor, so the parent's validation logic still runs. Then we added our own attribute (`isLED`) on top. We also overrode `calculateEnergyConsumption()` – our version first calls `super.calculateEnergyConsumption()` to get the base energy, and then applies a 20% discount if the light is LED. Our `CoolingAppliance` works the same way, but it adds a `temperatureSetting` and increases energy by 20% when the temperature is set below 24 degrees.

4.2.3 Polymorphism (Dynamic Behaviour)

From `Household.java` (entity) and `EnergyCalculationService.java` (service) – dynamic method dispatch using a superclass list:

```

// In Household.java (entity):
// The ArrayList uses the SUPERCLASS type (Appliance).
// It can hold Appliance, LightAppliance, or CoolingAppliance objects.
private ArrayList<Appliance> appliances;

// In EnergyCalculationService.java (service):
public double calculateTotalEnergy(Household household) {
    double totalEnergy = 0.0;

    for (int i = 0; i < household.getApplianceCount(); i++) {
        // UPCASTING: 'currentAppliance' is the superclass type (Appliance),
        // but the real object could be any subclass.
        Appliance currentAppliance = household.getAppliance(i);
    }
}

```

```

        // DYNAMIC DISPATCH: the JVM picks the right version of
        // calculateEnergyConsumption() based on the ACTUAL type at runtime.
        double energy = currentAppliance.calculateEnergyConsumption();

        totalEnergy = totalEnergy + energy;
    }

    return totalEnergy;
}

```

What this shows: This is where dynamic polymorphism really happens in our project. The `Household` entity stores all the appliances in one `ArrayList<Appliance>`, using the parent type – so when we add a `LightAppliance` or `CoolingAppliance` into this list, it gets **upcast** to `Appliance`. The `EnergyCalculationService` then loops over the list through the household’s encapsulated accessor methods (`getApplianceCount()` / `getAppliance(i)`), so the service never touches the private list directly.

Now, in the `for` loop, `currentAppliance` is declared as `Appliance`. But the actual object behind it could be anything – a `LightAppliance`, a `CoolingAppliance`, or a plain `Appliance`. When we call `currentAppliance.calculateEnergyConsumption()`, the JVM does not just run the parent’s version. It looks at what the object **actually** is at runtime and calls the right overridden method. This is called **dynamic method dispatch**.

So if the object is really a `LightAppliance` with LED on, the LED-discount version runs. If it is a `CoolingAppliance` below 24 degrees, the penalty version runs. We get different results from the same method call, and we did not need any `if-else` or `instanceof` to make this work. That is the whole point of polymorphism – one loop, one method call, different behaviour depending on the object.

If we ever add a new type of appliance (like a `HeatingAppliance`), we would not need to touch this loop. It would just pick up the new subclass’s method automatically.

4.3 Execution and User Input/Output Handling

Our controller class `EnergyTrackerController` handles all the user interaction. The entry point `EnergyTrackerApp.main()` creates the objects from the different classes of the application and wires them together before starting the menu loop:

From `EnergyTrackerApp.java` – `main()` creates objects from different classes (constructor injection):

```

public static void main(String[] args) {
    // main() creates one object from each layer of the application
    Household household = new Household("My Home"); //
    ↪ entity
    InputReader inputReader = new InputReader(); //
    ↪ utility
}

```

```

    EnergyCalculationService calculationService = new EnergyCalculationService();
↪ // service
    ReportService reportService = new ReportService(calculationService);    //
↪ service

    // CONSTRUCTOR INJECTION: the controller receives all its collaborators
    EnergyTrackerController controller = new EnergyTrackerController(
        household, inputReader, calculationService, reportService);

    controller.run(); // hand over control to the menu loop
}

```

Everything the user does from this point on – adding appliances, viewing reports – exercises object creation, getter/setter usage, inheritance, and polymorphic behaviour at runtime. The program shows a menu with four choices (add appliance, view report, view high-energy appliances, exit) and keeps looping until the user picks exit. All the fixed screens (welcome banner, menu, exit banner) are printed by the `ConsoleUI` utility class so the whole program has a consistent look.

How we handled bad input:

One thing we were careful about is making sure the program does not crash when the user types something wrong. We put all the input logic into a dedicated utility class, `InputReader`, which wraps the `java.util.Scanner` as a private field (Encapsulation) and provides methods like `readInt()`, `readDouble()`, `readYesNo()`, and `readNonEmptyString()` that use `try-catch` and `while` loops to keep asking until the user gives valid input. Because every part of the program reads input through this one class, the validation rules are guaranteed to be consistent everywhere.

From `InputReader.java` – handling invalid input with `try-catch`:

```

public int readInt(String prompt, int min, int max) {
    int value = 0;
    boolean validInput = false;

    while (!validInput) {
        System.out.print(" " + prompt);
        try {
            String input = scanner.nextLine().trim();
            value = Integer.parseInt(input);

            if (value >= min && value <= max) {
                validInput = true;
            } else {
                System.out.println(" [Invalid] Please enter a number between "
                    + min + " and " + max + ".");
            }
        } catch (NumberFormatException e) {
            System.out.println(" [Invalid] That is not a valid number. Please try
↪ again.");
        }
    }
}

```

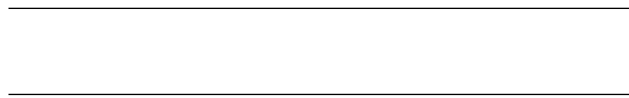
```

    return value;
}

```

What this shows: We used `Integer.parseInt()` inside a `try` block. If the user types something that is not a number (like “abc” or special characters), Java throws a `NumberFormatException`. We catch that error and just print a message telling the user to try again. The `while` loop keeps going until we get a valid number in the right range. We used the same idea for `readDouble()` and `readYesNo()`.

On top of that, we wrapped the controller’s entire `run()` menu loop in a `try-catch(Exception)` block with a `finally` part. The `finally` calls `inputReader.close()`, which makes sure the underlying `Scanner` always gets closed, even if something unexpected goes wrong. This way the program never just crashes on the user – it always shows a message and exits cleanly.



5 Output and Testing

5.1 Testing Strategy and Sample Inputs

Our team used **black-box testing** to test the program. What this means is that we tested it from the user’s point of view – we did not look at the internal code while testing. We just ran the program, typed in different inputs, and checked if the output matched what we expected.

We mainly focused on two things: - **Valid inputs** – to make sure the normal features work correctly (adding appliances, calculating energy, etc.). - **Invalid inputs** – to make sure the program does not crash when the user types something wrong (like letters instead of numbers, or negative values).

Each member designed and executed the test cases for the classes they implemented, so the testing effort is individual and traceable. Here are our ten main test cases:

Table 3: Test Cases {tbl-colwidths="[8, 25, 20, 25, 12, 10]"} Threshold: 50 kWh (same two appliances)
 | Report shows “No appliances exceed the energy threshold. Great job!” | MOHAMMAD SHAHNAWAZ | Pass |

Test ID	Description	Sample Input	Expected Outcome	Tested by	Pass/Fail
TC-01	Add a general appliance with valid data (full controller flow: menu -> input -> storage -> confirmation)	Name: “Ceiling Fan”, Power: 75 W, Duration: 8 h	Appliance added successfully. Energy = 0.6000 kWh.	Su Tingwei	Pass

Test ID	Description	Sample Input	Expected Outcome	Tested by	Pass/Fail
TC-02	Test input validation: enter a negative power rating	Power: -50	InputReader.readDouble() shows “[Invalid] Value cannot be negative. Please try again.” and keeps asking; the appliance is only created after a valid value is entered.	To Tsun Kai	Pass
TC-03	Test polymorphism: add an LED light and check energy discount	Name: “Desk Lamp”, Power: 60 W, Duration: 5 h, LED: yes	Energy = 0.2400 kWh (base 0.3000 x 0.8 = 20% LED discount applied by the overridden method).	ABDULLAH A. F. ZIM	Pass
TC-04	Test invalid input: type letters instead of a number for menu choice	Input: “abc” when asked for menu choice (1-4)	Program shows “[Invalid] That is not a valid number. Please try again.” and asks again. No crash.	To Tsun Kai	Pass
TC-05	Test summary calculations: add an LED lamp and an AC, then view the report to check total, average, and estimated cost	Desk Lamp (60 W, 5 h, LED) + Living Room AC (1500 W, 10 h, 22°C)	Total = 18.2400 kWh, Average = 9.1200 kWh/appliance, Estimated Cost = RM 3.98 (18.2400 × 0.218).	WEN SIHAO	Pass

Test ID	Description	Sample Input	Expected Outcome	Tested by	Pass/Fail
TC-06	Test polymorphism: cooling appliance below 24°C gets the 20% penalty	Name: “Aircon”, Power: 1000 W, Duration: 8 h, Temp: 20	Energy = 9.6000 kWh (base 8.0000 x 1.2 penalty applied by the overridden method).	ABDULLAH A. F. ZIM	Pass
TC-07	Test high-energy filter: threshold below one appliance’s usage	Threshold: 5 kWh (with Desk Lamp 0.24 kWh + Living Room AC 18.0 kWh added)	Report lists exactly 1 appliance: “Living Room AC — 18.0000 kWh”.	MOHAM-MAD SHAH-NAWAZ	Pass
TC-08	Test controller guard: view report before adding any appliance	Choose option 2 with an empty household	Program shows “[INFO] No appliances added yet. Please add appliances first (Option 1).” and returns to the menu.	Su Tingwei	Pass
TC-09	Test calculation edge case: zero power and zero duration	Name: “Standby Device”, Power: 0 W, Duration: 0 h	Appliance is accepted (0 is valid) and Energy = 0.0000 kWh – no division or formula error.	WEN SIHAO	Pass
TC-10	Test high-energy filter: threshold above every appliance’s usage				

All ten test cases passed.

5.2 Console Results (Sample Output)

Since we cannot attach screenshots in this format, we have copied the actual console output below.

Test Run 1 – Normal usage: adding appliances and viewing the energy report

```
=====
HOUSEHOLD ENERGY CONSUMPTION TRACKER
SDG 7: Clean Energy
-----

Track your appliances. Save energy. Save the planet.
=====

-----
MAIN MENU
-----

1. Add an Appliance
2. Record & View Total Energy Consumption
3. View High Energy Consumption Appliances
4. Exit Program
-----

Please enter your choice (1-4): 1

-----
ADD A NEW APPLIANCE
-----

Select appliance type:
1. General Appliance
2. Light Appliance
3. Cooling Appliance (e.g., Air Conditioner)
-----

Your choice (1-3): 2

-----

Enter appliance details:
-----

Appliance name: Desk Lamp
Power rating (Watts, >= 0): 60
Usage duration (Hours, >= 0): 5
Is this an LED light? (yes/no): yes

-----

[SUCCESS] Appliance added successfully!
-----

LightAppliance [Name: Desk Lamp, Power: 60.00 W, Duration: 5.00 h, LED: Yes,
↪ Energy: 0.2400 kWh]
Total appliances in household: 1
=====
```

```

-----
                        MAIN MENU
-----

1.  Add an Appliance
2.  Record & View Total Energy Consumption
3.  View High Energy Consumption Appliances
4.  Exit Program
-----

Please enter your choice (1-4): 1

-----

                        ADD A NEW APPLIANCE
-----

Select appliance type:
1.  General Appliance
2.  Light Appliance
3.  Cooling Appliance (e.g., Air Conditioner)
-----

Your choice (1-3): 3

-----

Enter appliance details:
-----

Appliance name: Living Room AC
Power rating (Watts, >= 0): 1500
Usage duration (Hours, >= 0): 10
Temperature setting (16-30 C): 22

-----

[SUCCESS] Appliance added successfully!
-----

CoolingAppliance [Name: Living Room AC, Power: 1500.00 W, Duration: 10.00 h,
↪ Temp: 22°C, Energy: 18.0000 kWh]
Total appliances in household: 2
-----

-----

                        MAIN MENU
-----

1.  Add an Appliance
2.  Record & View Total Energy Consumption
3.  View High Energy Consumption Appliances
4.  Exit Program
-----

Please enter your choice (1-4): 2

===== ENERGY CONSUMPTION REPORT =====
Household: My Home
Total Appliances: 2

```

--- Appliance #1 ---

Name : Desk Lamp
Power Rating : 60.00 W
Usage Duration : 5.00 h
Type : Light Appliance
LED : Yes (20% energy saving applied)
Energy Used : 0.2400 kWh

--- Appliance #2 ---

Name : Living Room AC
Power Rating : 1500.00 W
Usage Duration : 10.00 h
Type : Cooling Appliance
Temperature : 22 °C (20% energy penalty applied)
Energy Used : 18.0000 kWh

=====

SUMMARY REPORT

Total Energy Used : 18.2400 kWh
Average Usage : 9.1200 kWh/appliance
Estimated Cost : RM 3.98
(Tariff: RM 0.218 per kWh - TNB Domestic Rate)

=====

SDG 7 Reminder: Use LED lights and set your
air conditioner to 24°C or above to save energy!

=====

Test Run 2 – Invalid input handling: the program catches errors without crashing

MAIN MENU

1. Add an Appliance
2. Record & View Total Energy Consumption
3. View High Energy Consumption Appliances
4. Exit Program

Please enter your choice (1-4): abc
[Invalid] That is not a valid number. Please try again.
Please enter your choice (1-4): 5
[Invalid] Please enter a number between 1 and 4.
Please enter your choice (1-4): !@#
[Invalid] That is not a valid number. Please try again.
Please enter your choice (1-4): 1

ADD A NEW APPLIANCE

```

-----
Select appliance type:
1. General Appliance
2. Light Appliance
3. Cooling Appliance (e.g., Air Conditioner)
-----
Your choice (1-3): 1

-----
Enter appliance details:
-----
Appliance name:
[Invalid] Appliance name cannot be empty.
Appliance name: Test Fan
Power rating (Watts, >= 0): -200
[Invalid] Value cannot be negative. Please try again.
Power rating (Watts, >= 0): fifty
[Invalid] That is not a valid number. Please try again.
Power rating (Watts, >= 0): 50
Usage duration (Hours, >= 0): 3

-----
[SUCCESS] Appliance added successfully!
-----
Appliance [Name: Test Fan, Power: 50.00 W, Duration: 3.00 h, Energy: 0.1500
↵ kWh]
Total appliances in household: 1
-----

```

As shown above, the program handled every invalid input correctly. It did not crash at any point. It just told the user what went wrong and asked them to try again.

5.3 Issues, Limitations, and Debugging

During the development process, our team ran into a couple of bugs and limitations. Here is what happened and how we dealt with them.

5.3.1 Bug 1: The Scanner `.nextInt()` Skipping Problem

This was probably the most frustrating bug we encountered. Early in development, we were using `scanner.nextInt()` to read menu choices and `scanner.nextLine()` to read appliance names. But we noticed that after the user entered a number for the menu, the program would completely skip the next `nextLine()` call – it would just return an empty string without waiting for input.

After some research, we found out this is a well-known Java bug. When you call `scanner.nextInt()`, it reads the number but leaves the newline character (`\n`) in the input buffer. So when

`scanner.nextLine()` runs next, it immediately reads that leftover newline and returns an empty string.

How we fixed it: We decided to stop using `nextInt()` and `nextDouble()` entirely. Instead, we read everything as a string using `scanner.nextLine()`, and then convert it to a number using `Integer.parseInt()` or `Double.parseDouble()`. This way, the newline is always consumed, and we never have the skipping problem. It also made our `try-catch` error handling easier, because `parseInt()` throws a clean `NumberFormatException` that we can catch.

5.3.2 Bug 2: `toString()` Showing Wrong Energy for Subclasses

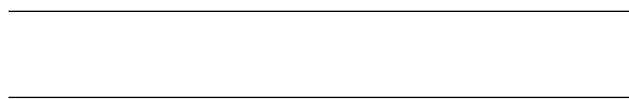
Another bug we found was that when we first wrote the `toString()` method in `LightAppliance`, we accidentally used `super.calculateEnergyConsumption()` in the format string instead of `calculateEnergyConsumption()`. This meant the printed energy value did not include the 20% LED discount, even though the actual calculation was correct internally. The numbers in the report were right, but the `toString()` output was showing the wrong value.

We caught this during testing when we compared the energy value printed by `toString()` with the value in the energy report. They did not match, so we traced it back to the `toString()` method and fixed it to call `this.calculateEnergyConsumption()` (or just `calculateEnergyConsumption()` without `super`).

5.3.3 Limitation: No Data Persistence

One limitation of our current program is that it does not save any data. Once the user closes the program, all the appliances and energy records are lost. Everything is stored in `ArrayList` objects in memory, so nothing survives after the program exits.

We are aware that this could be solved by using **File I/O** (writing data to a `.txt` or `.csv` file) or even connecting to a database. However, since we have not covered File I/O in this module yet, we decided to keep it simple for now. This is something we would like to improve in a future version of the project.



6 Discussion and Reflection

6.1 Reflection on OOP

Looking back, using OOP principles really did make our code easier to manage. One thing we noticed early on is that **Encapsulation** saved us a lot of trouble. Because we put validation checks inside our setter methods, the program automatically rejects bad data like negative power values or empty names. Without that, we would have had to write `if` checks everywhere in the controller, and it would have been easy to miss one.

Polymorphism was the part that impressed us the most. At first, we were not sure how useful it would be. But when we saw that our `EnergyCalculationService.calculateTotalEnergy()` method only needed **one for loop** to handle all three types of appliances – general, light, and cooling – it clicked. The JVM picks the right version of `calculateEnergyConsumption()` at runtime through dynamic method dispatch, so we did not need to write separate **if-else** blocks for each appliance type. And if we ever want to add a new type of appliance in the future (like a `HeatingAppliance`), we just create a new subclass and override the method. The existing loop does not need any changes at all. That is really convenient.

6.2 What We Learned

Besides the OOP concepts, our team also learned some practical things during this project. The biggest one was dealing with the `Scanner` bug – when `nextInt()` leaves a newline in the buffer and causes `nextLine()` to skip. It took us a while to figure out what was going on, but once we switched to using `nextLine()` with `Integer.parseInt()`, the problem went away completely. It was a good reminder that debugging is not just about logic errors – sometimes the tools themselves behave in unexpected ways.

We also learned a lot about working as a team. At the start, we had some confusion about who was writing which class, and there were a few merge conflicts on GitHub. This is actually one of the reasons we refactored the project into a layered architecture with 11 small classes: once we split the work by layer (entity, service, controller, utilities – each class owned by exactly one member, see `TEAM_CONTRIBUTIONS.md`), nobody had to edit the same file as someone else, and things went much smoother. Using Git helped us track changes and avoid overwriting each other’s code.

6.3 Future Improvements

There are two main things we would like to add if we continue working on this project:

1. **Data persistence with File I/O** – Right now, all the data disappears when the program closes. In the future, we want to save the appliance list to a `.txt` or `.csv` file so that users can load their data the next time they open the program. We have not learned File I/O yet in this module, but it is something we plan to study on our own.
2. **Graphical User Interface (GUI)** – Our current program runs entirely in the console with text menus. While it works fine, it is not very user-friendly for non-technical users. In the future, we would like to replace the console interface with a simple GUI using **JavaFX**. This would make the program look more professional and easier to use, with buttons, text fields, and maybe even a chart to visualize energy consumption.



7 Conclusion

In conclusion, our team successfully developed a Household Energy Consumption Tracker using Java and core OOP principles. The program allows users to add different types of appliances, calculate their individual and total energy consumption, and identify which appliances are using the most electricity. We applied **Encapsulation** to protect data with validated setters, **Inheritance** to avoid repeating code across appliance types, and **Polymorphism** to let a single loop handle different appliance calculations through dynamic method dispatch. The program also handles invalid user input without crashing, which was one of our key goals.

This project is directly linked to **SDG 7: Affordable and Clean Energy**. By helping users see exactly how much energy each appliance uses, our program encourages them to switch to energy-efficient options like LED lights and to avoid setting their air conditioners too low. This reduces unnecessary energy waste at the household level.

Beyond SDG 7, our project also connects to **SDG 3: Good Health and Well-being**. In a hot country like Malaysia, cooling appliances like fans and air conditioners are not just about comfort – they are about health. Extreme heat can cause heatstroke and other heat-related illnesses, especially for elderly family members and young children. By helping households manage their cooling appliances more efficiently, our program makes sure families can keep their homes at a safe and comfortable temperature without wasting energy. In other words, saving energy does not mean turning off the AC completely – it means using it wisely. And that supports both clean energy and family health at the same time.

8 References

Liang, Y. D. (2020). *Introduction to Java programming and data structures: Comprehensive version* (12th ed.). Pearson.

Oracle. (2014). *Java Platform, Standard Edition 8 API Specification*. Oracle Corporation. <https://docs.oracle.com/javase/8/docs/api/>

United Nations. (n.d.). *Goal 7: Ensure access to affordable, reliable, sustainable and modern energy for all*. United Nations Department of Economic and Social Affairs, Sustainable Development. <https://sdgs.un.org/goals/goal7>

Oracle. (n.d.). *The Java™ tutorials: Object-oriented programming concepts*. Oracle Corporation. <https://docs.oracle.com/javase/tutorial/java/concepts/>

Tenaga Nasional Berhad. (n.d.). *Pricing & tariffs: Residential*. <https://www.tnb.com.my/residential/pricing-tariffs>

United Nations. (n.d.). *Goal 3: Ensure healthy lives and promote well-being for all at all ages*. United Nations Department of Economic and Social Affairs, Sustainable Development. <https://sdgs.un.org/goals/goal3>